

DevOps Task Manager Deployment Project

End-to-End CI/CD | Infrastructure as Code | Cloud Deployment | Kubernetes | Monitoring

1. Project Overview

This project demonstrates a complete end-to-end DevOps implementation using industry-standard cloud technologies and automation tools. The application is a React-based Task Manager that is fully automated from source code to cloud deployment and real-time monitoring.

The primary goal is to simulate a real-world DevOps workflow where every step—from a developer pushing code, to the application being live on the cloud—is handled automatically without manual intervention.

Key Capabilities

- **Automated Infrastructure Provisioning** using Terraform to create AWS EC2 instances, security groups, and required networking.
- **CI/CD Pipeline** using Jenkins with automated triggers and sequential stage execution.
- **Containerization** using Docker with optimized multi-stage builds to ensure a small footprint.
- **Container Orchestration** using K3s (Lightweight Kubernetes) for self-healing and high-availability application hosting.
- **Cloud Deployment** directly on AWS EC2, configured securely for automated pipeline access.
- **Real-Time Monitoring** with Prometheus and Grafana, capturing vital system and application metrics dynamically.

2. Technologies Used

The following tools and technologies form the core of this project. Each tool plays a specific role in the automation pipeline.

Tool / Technology	Purpose
Git & GitHub	Distributed version control and cloud-based repository hosting for tracking source code changes.
Docker	Containerization platform for packaging the React application and its dependencies into isolated images.
Jenkins	Open-source CI/CD server for defining and automating the pipeline workflow.
Terraform	Infrastructure as Code (IaC) tool for provisioning, managing, and destroying AWS cloud resources.
AWS EC2	Cloud virtual machine to host the application, Kubernetes cluster, and monitoring stack.
K3s	Lightweight, CNCF-certified Kubernetes distribution tailored for resource-constrained orchestration.
Prometheus	Time-series database, metrics collection, and alerting engine.
Grafana	Interactive, real-time visualization dashboard connected to Prometheus data sources.

3. Project Architecture

The architecture is deliberately split into two distinct phases to ensure stability and efficiency: a **One-Time Manual Setup** for infrastructure, and an automated **Per-Push CI/CD Pipeline** for application updates.

Phase 1: One-Time Manual Setup

This phase is executed locally by the developer only once to bootstrap the environment.

- **Step 1 — Terraform Provisioning:** The developer runs `terraform apply` locally. This creates the AWS EC2 instance, security groups, and automatically installs Docker and K3s on boot.
- **Step 2 — Verify Infrastructure:** The developer SSHs into the EC2 instance to verify the Kubernetes node is Ready.
- **Step 3 — Deploy Monitoring:** The developer manually applies the Prometheus and Grafana manifests to the cluster.

Phase 2: Automated CI/CD Pipeline (Jenkins)

This is the automated workflow triggered on every code push to GitHub.

- **Step 1 — Trigger:** GitHub webhook notifies Jenkins of a new commit.
- **Step 2 — Checkout & Build:** Jenkins checks out the code and builds a highly optimized Docker image of the React application.
- **Step 3 — Push to Registry:** Jenkins pushes the new image to DockerHub.
- **Step 4 — SSH & Deploy:** Jenkins dynamically fetches the running EC2 IP using the AWS CLI, connects via SSH, and applies the updated Kubernetes application manifests. The application updates live immediately without recreating infrastructure.

4. Project Structure

Below is the complete folder structure of the project repository, organized by functional boundaries:

```
devops-task-manager/
├── app/                # React frontend source code (Vite + React)
├── docker/             # Dockerfile for multi-stage image build
├── k8s/                # Kubernetes Deployment and Service manifests
├── terraform/          # AWS Infrastructure as Code (provisions K3s & EC2)
├── jenkins/            # Jenkinsfile for CI/CD pipeline definition
└── monitoring/         # Prometheus + Grafana configuration manifests
```

5. Prerequisites & Local Setup

Before starting, the following tools must be installed on your local development machine:

5.1 Install Git

```
# Windows (via Chocolatey)
choco install git
# Verify
git --version
```

5.2 Install Docker

Download and install Docker Desktop from the official site: [Docker Desktop \(https://www.docker.com/products/docker-desktop/\)](https://www.docker.com/products/docker-desktop/)

```
docker --version
```

5.3 Install Terraform

```
# Windows (via Chocolatey)
choco install terraform
# Verify
terraform -version
```

5.4 Configure AWS CLI

Download and install the AWS CLI.

```
aws configure
# Enter: AWS Access Key, AWS Secret Key, Region, Output format (json)
```

6. Docker — Containerization

Docker is used to package the React Task Manager into a portable container image. A multi-stage build is used to keep the final image extremely small, secure, and production-ready by discarding the Node.js build environment and only keeping compiled static files served by Nginx.

Dockerfile snippet (docker/Dockerfile)

```
# Stage 1: Build the React application
FROM node:20-alpine as build

WORKDIR /app

# Copy package.json and install dependencies securely
COPY app/package.json app/package-lock.json* ./
RUN npm install

# Copy the rest of the application code and build
COPY app/ ./
RUN npm run build

# Stage 2: Serve the application with Nginx (Production Stage)
FROM nginx:alpine

# Remove default nginx static assets for a clean slate
RUN rm -rf /usr/share/nginx/html/*

# Copy only the compiled assets from the builder stage
COPY --from=build /app/dist /usr/share/nginx/html

# Expose HTTP port 80
EXPOSE 80

# Start Nginx in the foreground
CMD ["nginx", "-g", "daemon off;"]
```

Build and Push Commands

```
docker build -t your-docker-id/task-manager:latest -f docker/Dockerfile .
docker push your-docker-id/task-manager:latest
```

7. Terraform — Infrastructure as Code

Terraform is used to define and provision AWS cloud infrastructure in a declarative, version-controlled manner. This entirely eliminates manual AWS Console operations and guarantees reproducible environments.

Resources Provisioned

- **AWS EC2 Instance** (Ubuntu, t3.micro or t2.small).
- **Security Group** with exact inbound/outbound rules required for SSH, HTTP, and Kubernetes services.
- **Automated Bootstrap** via `user_data` script to install Docker and K3s silently on first boot.

Terraform Workflow (One-Time Manual Setup)

```
cd terraform
terraform init # Download AWS providers and modules
terraform apply -auto-approve # Provision infrastructure on AWS
```

Security Group Port Configuration

Port	Protocol	Purpose / Service
------	----------	-------------------

22	TCP	SSH — Remote access for Jenkins and debugging
80	TCP	HTTP — Node.js / Nginx general web traffic
6443	TCP	Kubernetes API — Administrative K3s access
30000 - 32767	TCP	NodePort Range — Required for exposed K8s Services

Terraform Configuration Snippet (terraform/main.tf)

```
# Security Group allowing specific inbound traffic
resource "aws_security_group" "k3s_sg" {
  name        = "k3s_node_sg"
  description = "Allow inbound traffic for k3s, HTTP, and SSH"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "NodePort Service Range"
    from_port   = 30000
    to_port     = 32767
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Outbound rule for all traffic
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# EC2 Instance configured to auto-install Docker and K3s
resource "aws_instance" "k3s_node" {
  ami           = var.ami_id
  instance_type = var.instance_type
  key_name      = var.key_name
  vpc_security_group_ids = [aws_security_group.k3s_sg.id]

  # User data executes automatically as root when the instance boots
  user_data = <<-EOF
    #!/bin/bash
    set -ex
    apt-get update -y
    # Docker installation script...
    apt-get install -y docker-ce docker-ce-cli containerd.io
    # K3s installation script...
    curl -sLf https://get.k3s.io | sh -s - --write-kubeconfig-mode 644 --node-external-ip=$(curl -s
http://169.254.169.254/latest/meta-data/public-ipv4)
  EOF
}
```

8. Jenkins — CI/CD Pipeline

Jenkins serves as the automation backbone of this project. The pipeline is defined in a `Jenkinsfile` committed to the repository, enabling pipeline-as-code principles.

Pipeline Stages

1. **Checkout:** Pull the latest source code from the GitHub repository.
2. **Build Docker Image:** The React app is built into a Docker image using the provided multi-stage Dockerfile.
3. **Push to Docker Hub:** The newly built container image is pushed to a public or private Docker registry.
4. **Deploy Application to K3s:** Using the AWS CLI, Jenkins fetches the IP of the existing EC2 instance. It then securely connects via SSH and applies the updated Kubernetes application manifests dynamically.

9. Kubernetes with K3s

K3s is a lightweight, CNCF-certified Kubernetes distribution designed specifically for resource-constrained environments. It provides full Kubernetes functionality while running efficiently on a standard `t3.micro` EC2 instance.

Why K3s?

- Significantly lower memory footprint compared to full Kubernetes (EKS/kubeadm).
- Single binary installation with no complex setup.
- Full Kubernetes API compatibility and built-in self-healing capabilities.

Deployment Manifest (k8s/deployment.yaml)

This configuration tells Kubernetes to maintain exactly two instances (replicas) of the Task Manager app at all times.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-manager-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: task-manager
  template:
    metadata:
      labels:
        app: task-manager
    spec:
      containers:
        - name: task-manager
          image: komalpreet1703/task-manager:latest
          ports:
            - containerPort: 80
```

Service Manifest (k8s/service.yaml)

The Service exposes the internal Deployment to the outside world using a NodePort, allowing web traffic to reach the app.

```
apiVersion: v1
kind: Service
metadata:
  name: task-manager-service
spec:
  type: NodePort
  selector:
    app: task-manager
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

Verify the Kubernetes Cluster

Connect via SSH to your newly created instance and run these commands to verify health:

```
ssh -i key.pem ubuntu@<EC2-PUBLIC-IP>
sudo kubectl get nodes
sudo kubectl get pods -A
sudo kubectl get svc
```

10. Monitoring & Observability

The monitoring stack provides real-time visibility into the health and performance of the Kubernetes cluster.

- **Prometheus:** Automatically deployed via Kubernetes manifests. It scrapes cluster metrics, pod statistics, and node information, storing it in a high-performance time-series database.
- **Grafana:** Acts as the visual frontend. It connects to Prometheus as a data source and renders metrics (like CPU usage, memory limits, network I/O) into comprehensive, interactive dashboards.

Deployed natively to K3s using:

```
kubectl apply -f monitoring/prometheus.yaml
kubectl apply -f monitoring/grafana.yaml
```

11. Verification & Access URLs

Once the Jenkins pipeline completes successfully, all systems are immediately available over the public internet via the newly provisioned EC2 instance.

Service	URL / Access Path	Default Credentials
Task Manager App	http://<EC2_PUBLIC_IP>:30080	N/A
Prometheus Metrics	http://<EC2_PUBLIC_IP>:30090	N/A
Grafana Dashboard	http://<EC2_PUBLIC_IP>:30030	admin / admin123

12. Challenges & Solutions

Several real-world DevOps challenges were encountered and successfully resolved during implementation:

Challenge	Solution
Terraform AWS Credentials in Pipeline	Implemented the Jenkins <code>withCredentials</code> block to securely inject AWS Access Keys during pipeline execution, preventing hardcoded secrets.
EC2 SSH Access in Jenkins	Leveraged the <code>sshUserPrivateKey</code> plugin and added <code>StrictHostKeyChecking=no</code> to bypass initial prompt blockages during automated connections.
K3s Initialization Delay	Added an intelligent <code>sleep</code> and <code>retry</code> mechanism in Jenkins to verify the EC2 instance and K3s API were fully responsive before deploying manifests.
Dynamic Docker Image Tagging	Integrated an automated <code>shell</code> command <code>step</code> to swap out placeholder image tags in <code>deployment.yaml</code> with the latest build tags just prior to applying.

13. Conclusion

This project successfully establishes a professional, real-world DevOps automation workflow. Starting from a modern React Task Manager application, the project implements critical layers of cloud-native development:

- Source Control Integration
- Automated CI/CD Pipelines
- Infrastructure as Code (IaC)
- Optimized Containerization
- Kubernetes Orchestration
- Deep Observability & Monitoring

By completely automating the deployment cycle from Git push to live cloud monitoring, this project serves as a highly robust demonstration of enterprise-grade DevOps practices.